

Technical Decisions

These are proposed decisions for the later implementation. Record deviations in the final README.

ADR-001: Use PostgreSQL as the concurrency authority

Status: proposed.

Decision: use PostgreSQL and make the database transaction, not application memory, authoritative for capacity and duplicate protection.

Why: the task explicitly tests concurrent correctness. PostgreSQL provides row locks, partial unique indexes, and realistic behavior across server instances.

Tradeoff: Docker or a hosted database adds setup time. Mitigate with Docker Compose, migrations, seed data, and concise commands.

ADR-002: Serialize confirmations by locking the class row

Status: proposed.

Decision: successful payment handling locks the selected `trial_classes` row, re-counts confirmed bookings, and updates the booking within one transaction.

Why: it makes the last-seat rule easy to reason about and test. Different classes can still confirm independently.

Tradeoff: confirmations for the same class are serialized. With a maximum of four students this contention is negligible.

ADR-003: Only confirmed bookings consume capacity

Status: proposed.

Decision: selection and `pending_payment` do not reserve a seat.

Why: it matches the required scenario and avoids abandoned checkout holds.

Tradeoff: a user can reach payment and then lose the seat. The UI must state that availability is not guaranteed until confirmation.

ADR-004: Separate bookings from payment attempts

Status: proposed.

Decision: store payment attempts in their own table and keep booking status as the enrollment outcome.

Why: retries and failures are auditable, and a successful payment does not incorrectly imply a confirmed seat.

Tradeoff: one additional table and workflow branch.

ADR-005: Use idempotency keys on command endpoints

Status: proposed.

Decision: booking creation and payment-result recording require stable idempotency keys.

Why: double-clicks and provider retries are normal. Retry safety is part of reliability even though the brief does not name it directly.

Tradeoff: key storage and conflicting-payload validation require a small amount of extra code.

ADR-006: Prevent one student from holding multiple active bookings

Status: proposed.

Decision: a partial unique index covers `(student_id, trial_class_id)` while status is `pending_payment` or `confirmed`.

Why: preventing only duplicate confirmed rows still allows multiple simultaneous checkouts and possible duplicate charges.

Tradeoff: a failed booking must leave the active set before the parent can retry. This is intentional.

ADR-007: Treat frontend availability checks as advisory

Status: proposed.

Decision: the UI may hide full classes or disable buttons, but the backend always revalidates and the database decides.

Why: browser state is stale and untrusted.

Tradeoff: the user may receive a conflict after seeing availability. Return an explicit, friendly status.

ADR-008: Keep the frontend minimal and server-led

Status: proposed.

Decision: build one compact flow for selection, booking, payment simulation, status, and roster.

Why: reviewers prioritize backend judgment, verification, scope control, and explanation.

Tradeoff: the demo will look intentionally plain. Spend remaining time on clarity rather than visual systems.

ADR-009: Use authorization/capture as the production evolution

Status: proposed for documentation, not the mock implementation.

Decision: describe a production payment flow that authorizes funds, atomically claims the seat, then captures; void on capacity loss.

Why: an external provider cannot be committed atomically with a local database.

Tradeoff: capture failure after seat claim still needs reconciliation. That workflow is deliberately outside the take-home implementation.

ADR-010: Use compact spec-driven development

Status: accepted for planning.

Decision: maintain a small chain from requirements and acceptance criteria to technical design, ordered tasks, and verification cases before implementation begins.

Why: the brief rewards correctness, communication, and scope control. Traceability reduces the chance that a required edge case is lost during the timebox.

Tradeoff: documents can become overhead. Keep them concise, prepare them before the implementation timer, and update only when a decision or behavior changes.

ADR-011: Do not implement a distributed Saga in the take-home

Status: proposed.

Decision: use a PostgreSQL transaction and booking state machine for the implemented mock-payment flow. Describe Saga-like compensation only as the production evolution.

Why: the current solution has one authoritative transactional database boundary. A Saga is justified when payment, booking, refund, or notification operations commit independently.

Tradeoff: the mock does not demonstrate a complete distributed recovery workflow. The README must explain authorization, capture, void/refund, idempotency, and reconciliation accurately.

ADR-012: Do not introduce Kafka without demonstrated requirements

Status: proposed.

Decision: omit Kafka from the take-home runtime. If reliable asynchronous publication becomes necessary, begin with a transactional outbox and a simple publisher abstraction.

Why: the brief does not require high throughput, durable replay, or multiple independent consumers. Kafka would add setup and failure modes without improving the required booking invariants.

Tradeoff: asynchronous integrations are documented rather than demonstrated. Reconsider Kafka when measured volume, replay, ordering, retention, and consumer-independence requirements justify its operations.

ADR-013: Use a short-lived feature branch instead of full GitFlow

Status: proposed.

Decision: develop on `feature/trial-booking`, make coherent commits, and merge once into `main` through a pull request.

Why: the repository represents one timeboxed outcome with no parallel release, maintenance, or hotfix streams.

Tradeoff: this does not demonstrate every GitFlow branch type. It demonstrates a more important skill for this exercise: selecting the lightest workflow that preserves reviewability.